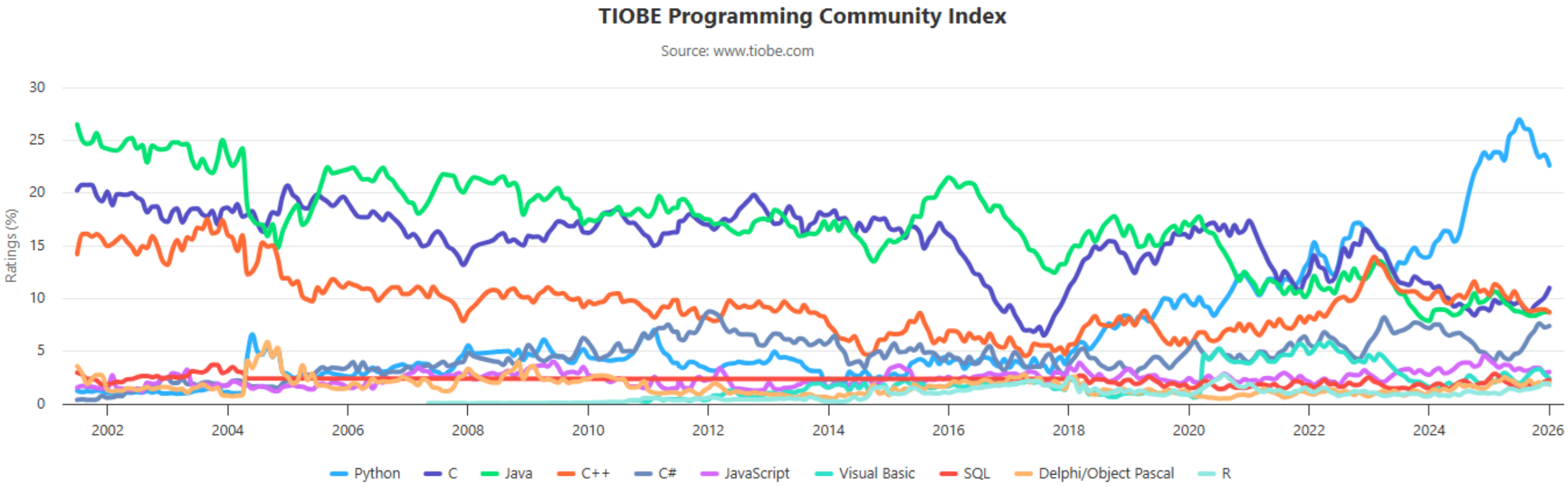
UET.CS2043 - Lập trình nâng cao

Hệ sinh thái Java & Tư duy hướng đối tượng


TIOBE Programming Community Index
Source: www.tiobe.com
Ratings (%)
30
25
20
15
10
5
0
2002
2004
2006
2008
2010
2012
2014
2016
2018
2020
2022
2024
2026
Python
C
Java
C++
C#
JavaScript
Visual Basic
SQL
Delphi/Object Pascal
R

History of Programming Languages

1950s	1960s	1970s	1980s	1990s	2000s	2010s
• Machine Code	• FORTRAN	• C	• C++	• Java	• C#	• Go
• *Assembly Language*	• COBOL	• Pascal	• Ada	• JavaScript	• Ruby	• Rust
	• Lisp	• BASIC	• Perl	• PHP	• Python	• Julia

Java

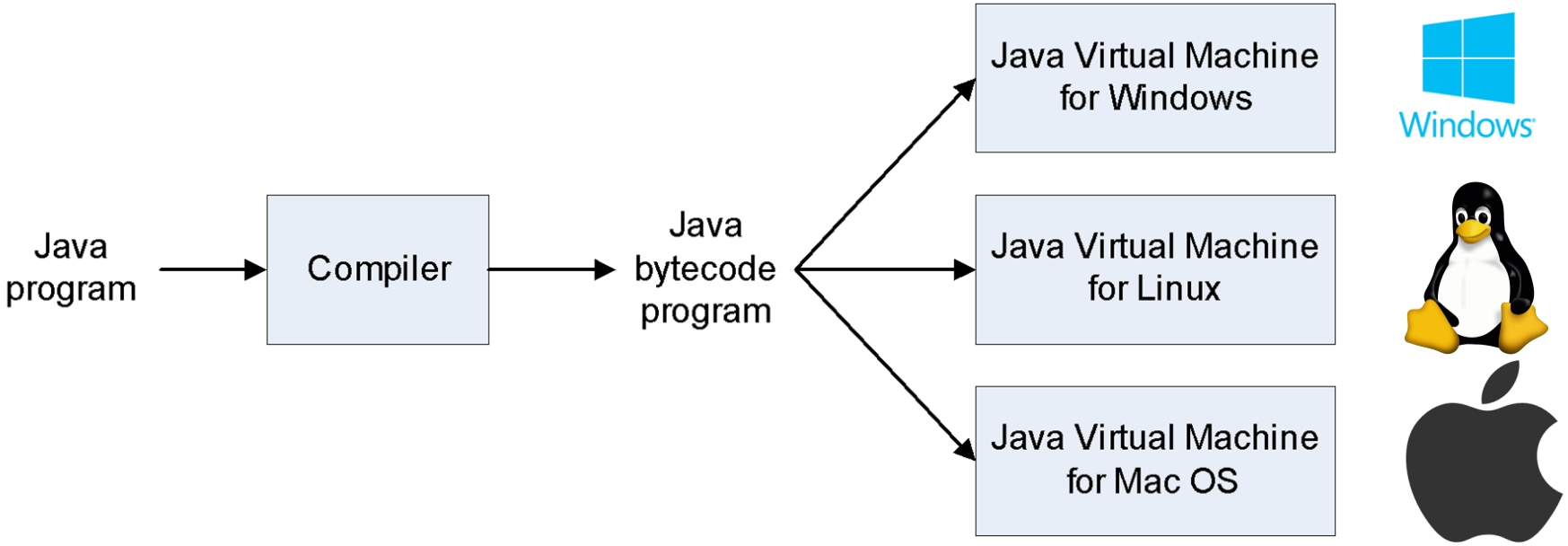
- 1991: phát triển bởi Sun Microsystems như là một ngôn ngữ lập trình cho môi trường nhúng
  - Tên ban đầu là Oak
- Java 1.0.2, 1.1
  - “Write Once, Run Anywhere”
  - Chậm
  - Được sử dụng cho các ứng dụng Web (applets)
- Java 2 (versions 1.2 - 1.4)
  - Nhanh và mạnh hơn,
  - 3 nền tảng: J2ME, J2SE, J2EE
- Java 5, 6, 7, 8 (versions 1.5-1.6…)
  - Được cải tiến nhiều hơn nữa

Cấu trúc của chương trình Java

- Tập hợp của các lớp đối tượng
- Thông thường mỗi lớp được mô tả trong 1 tệp mã nguồn (trùng tên với tên lớp)
  - Tăng tính độc lập
  - Dễ sửa đổi, tiết kiệm thời gian biên dịch

Biên dịch

- Mã nguồn Java được biên dịch thành mã bytecode
- Bytecode độc lập với nền tảng
- Bytecode được thực thi qua máy ảo Java (virtual machine)



Biên dịch

- Write Once, Run Anywhere

Java Virtual Machine (JVM)

- JVM được hiện thực phụ thuộc vào các nền tảng (hardware, OS)
- Đảm bảo các chương trình Java (bytecode) có thể thực thi trên các nền tảng khác nhau (platform-independent)
- Đảm bảo an ninh
- Thường được hiện thực như là một phần mềm
  - JRE - Java Runtime Environment
- Java platform: JVM + APIs

Các loại ứng dụng Java

- Ứng dụng để bàn – Java Standard Edition (Java SE)
- Ứng dụng phân tán, ứng dụng chủ – Java Enterprise Edition (Java EE)
- Ứng dụng di động – Java Micro Edition (Java ME)
- Ứng dụng trên thẻ - Java Card

HelloWorld application

trùng tên lớp

HelloWorld.java:

Lớp *Tên lớp*

```
public class HelloWorld {

  public static void main (String[] args) {

    System.out.println("Hello, world");

  }
}
```

Tên phương thức

Phát biểu trong phương thức

public, từ khóa quy định mức truy xuất

Biên dịch và thực thi

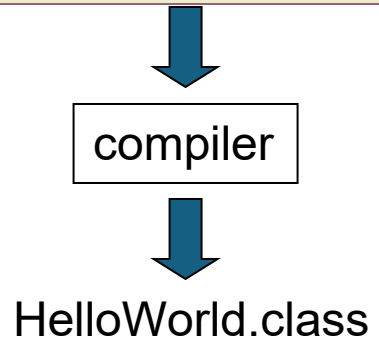
- Compile HelloWorld.java

  javac HelloWorld.java

- Run

  java HelloWorld

```
public class HelloWorld {
  public static void main (String[] args)
  {
    System.out.println("Hello, world");
  }
}
```




```
%> javac HelloWorld.java
%> java HelloWorld
Hello, world
```

Ứng dụng với nhiều hơn một lớp

2 lớp được hiện thực bởi 2 file

- TestGreeting.java:

```
public class TestGreeting {
  public static void main(String[] args) {
    Greeting gr = new Greeting();
    gr.greet();
  }
}
```

- Greeting.java:

```
public class Greeting {
  public void greet() {
    System.out.print("Hi there!");
  }
}
```

Biên dịch và thực thi

- Compile

javac TestGreeting.java

  - Greeting.java được dịch tự động

- Run

java TestGreeting

```
%> javac TestGreeting.java
%> java TestGreeting
Hi there!
```

JDK – Java Development Kit

- Môi trường phát triển ứng dụng Java
- Các thành phần chính
  - *javac*: trình biên dịch, chuyển mã nguồn Java thành bytecode
  - *java*: trình thông dịch
  - *javadoc*: công cụ sinh tài liệu từ các chú thích trong mã nguồn
  - *jdb*: trình gỡ lỗi

Lập trình hướng đối tượng (HĐT)

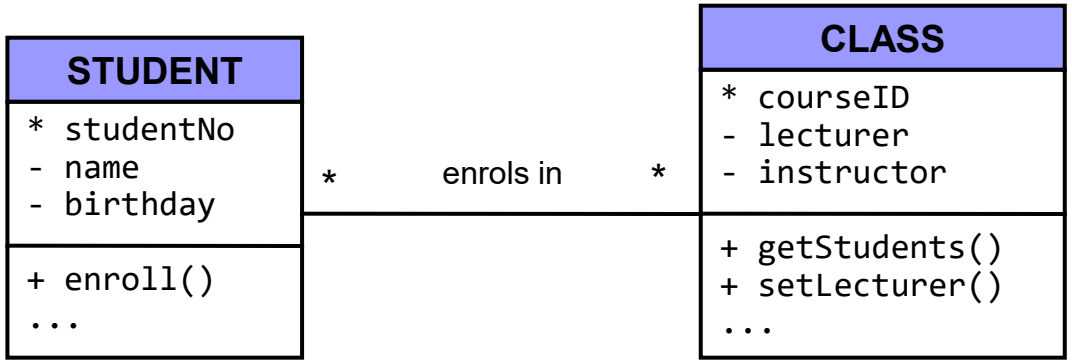
- Khắc phục các hạn chế của Lập trình thủ tục
  - Đóng gói dữ liệu và xử lý (che giấu dữ liệu; truy cập qua giao diện)
  - Đảm bảo nhất quán dữ liệu và các ràng buộc
  - Dễ bảo trì hơn

```
class Date{
public:
   void setDate(int newDay, int newMonth, int newYear);
   int getDay() { return day; } ...
private:
   int day;
   int month;
   int year;
};

void Date::setDate(int newDay, int newMonth, int newYear){
   //check validity of newDay, newMonth, newYear
   ...
   //set new values
   ...
}
```

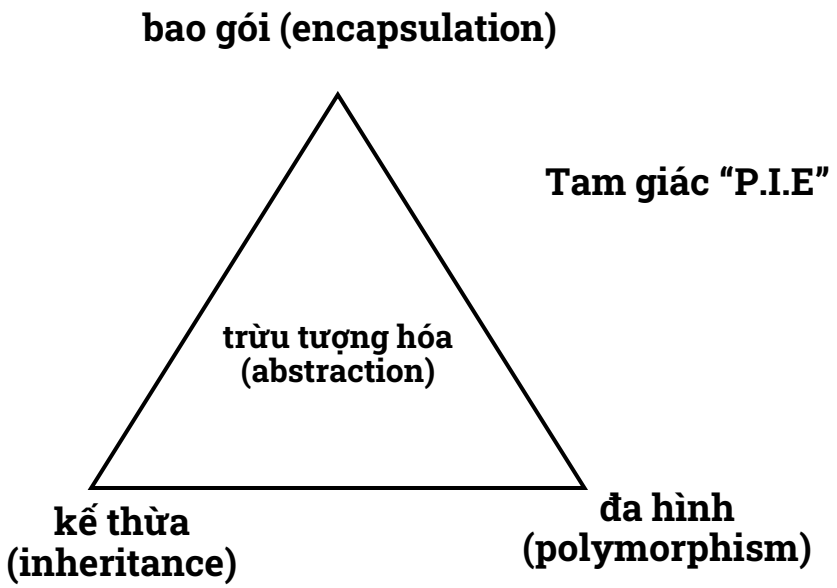
Lập trình HĐT (OOP) là gì?

- Lập trình hướng đối tượng
  - Tái hiện / ánh xạ vấn đề trong thế giới thực vào các chương trình
  - Xây dựng cơ chế tổ chức các "thực thể" (đối tượng) có thể làm gì đó hoặc yêu cầu các đối tượng khác thực hiện
  - Tạo ra "kiểu" (lớp) của các đối tượng để cho phép tạo ra chúng mà không phải định nghĩa lại thuộc tính và hành vi cho từng đối tượng



Khái niệm cơ bản về HĐT

- Trừu tượng hóa
- Các đối tượng & lớp
  - Trạng thái và hành vi đối tượng
  - Định danh đối tượng
  - Các thông điệp
- Bao gói
  - Che giấu thông tin (cài đặt)
- Kế thừa
- Đa hình



Trừu tượng hóa

- Đặc điểm của ngôn ngữ hướng đối tượng (khởi đầu bởi Smalltalk)
  - Mọi thứ đều là đối tượng (everything is an object)
  - Hoạt động của chương trình là quá trình “nói chuyện” giữa các đối tượng bằng cách “gửi thông điệp”
  - Mỗi đối tượng có vùng nhớ riêng và được hình thành từ các đối tượng khác
  - Mỗi đối tượng có một kiểu (type)
  - Các đối tượng cùng kiểu có thể nhận cùng một thông điệp

Các đối tượng

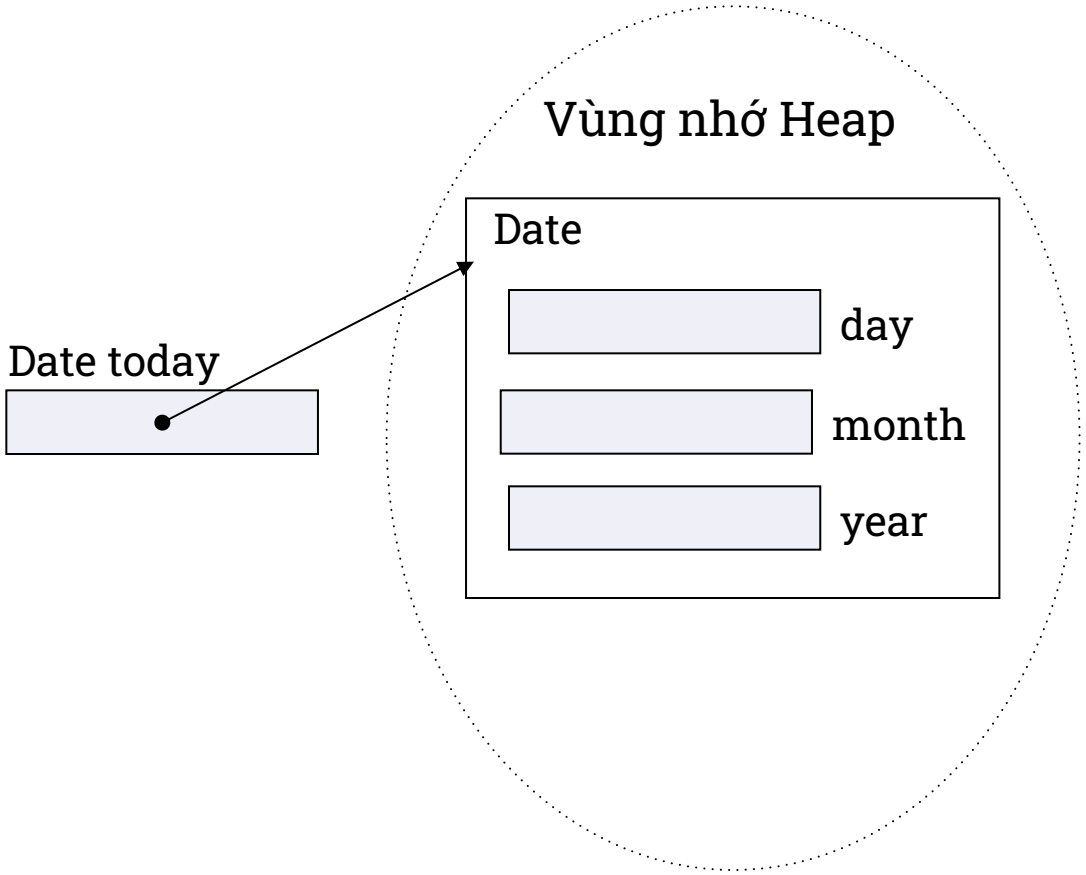
- Bao gồm ba thành phần
  - Trạng thái
    - Được xác định bởi dữ liệu bên trong (giá trị thuộc tính)
  - Hành vi
    - Tương ứng với các phương thức
  - Định danh
    - Các đối tượng ở bất kỳ trạng thái nào luôn có định danh duy nhất
      - Các đối tượng khác nhau có thể cùng trạng thái (cùng giá trị thuộc tính) nhưng khác nhau về định danh
      - Đối tượng được thao tác thông qua tham chiếu (handle) (con trỏ trong C++; tham chiếu đối tượng trong Java)

```
class Date{
public:
   void setDate(…);
   int getDay();
   ...
private:
   int day;
   int month;
   int year;
};

Date today = new Date();

today.setDate(2, 9, 2020);
```

Đối tượng và Tham chiếu đối tượng



Các thông điệp (messages)

today.setDate(2, 9, 2020);

- Là cách thức để đối tượng A yêu cầu đối tượng B thực hiện phương thức của B
- Một thông điệp bao gồm
  - Tham chiếu đối tượng đích - nhận thông điệp (today)
  - Tên của phương thức cần thực hiện (setDate)
  - Các thông tin cần thiết khác - các tham số (2, 9, 2020)
- Thông điệp được cài đặt ở dạng lời gọi hàm với tham số ngầm là đối tượng nhận thông điệp (method invocation)
- Khái niệm thông điệp tạo nên sự khác biệt cho lập trình HĐT
  - Dữ liệu trở thành chủ động
  - Với các lược đồ lập trình khác
    - Các hành động được thực hiện qua cơ chế truyền điều khiển (gọi hàm/thủ tục)
    - Dữ liệu là bị động; các thủ tục thao tác trên dữ liệu

Các lớp đối tượng

- Lớp (Class)
  - Khuôn để tạo các đối tượng (các thể hiện)
  - Các đối tượng của một lớp có cùng cấu trúc và hành vi được mô tả bởi lớp đó
- Quan hệ "Kiểu dữ liệu – Biến"
  - Các lớp được thiết kế và cài đặt (kiểu dữ liệu)
  - Các đối tượng được tạo khi thực thi chương trình

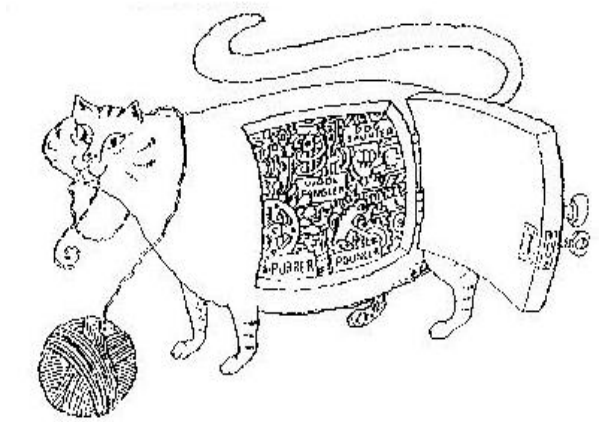
```
class Date{
public:
   void setDate(…);
   ...
private:
   int day;
   ...
};

Date today = new Date();

today.setDate(2, 9, 2020);
```

Bao gói / Che giấu thông tin

- Bao gói (encapsulation): để nhóm các thứ liên quan thành một tổng thể có định danh
  - Hàm/Thủ tục bao gói các câu lệnh
  - Đối tượng bao gói dữ liệu và thủ tục
- Che giấu thông tin: để che giấu các thông tin và chi tiết cài đặt bên trong đối với bên ngoài
  - Để các “clients” (các lập trình viên khác) xem các đối tượng như các hộp đen
  - public, private, và protected
  - Giao diện (interface) & Cài đặt (implementation)

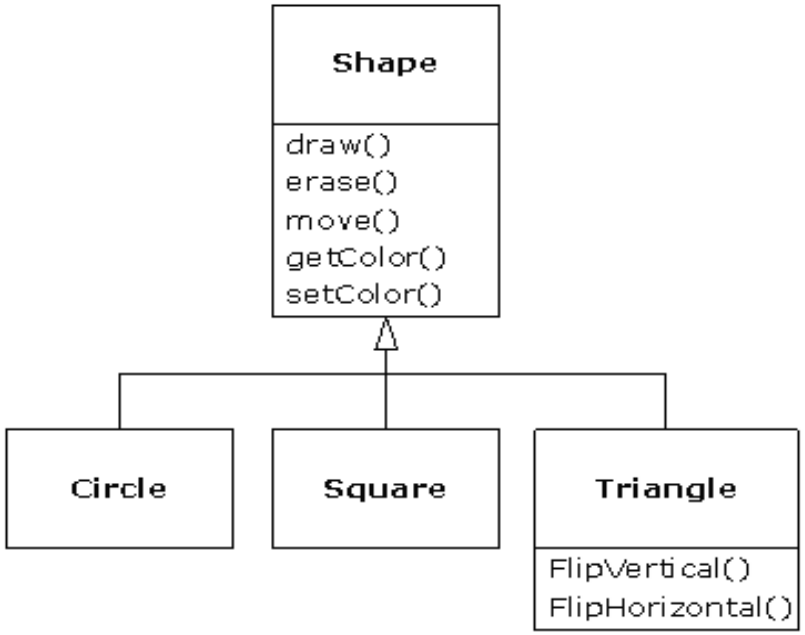


```
class Date{
public:
   void setDate(…);
   int getDay()
...
private:
   int day;
   int month;
   int year;
};

void Date::setDate{
   //check validity
   ...
   //set new values
   ...
}
```

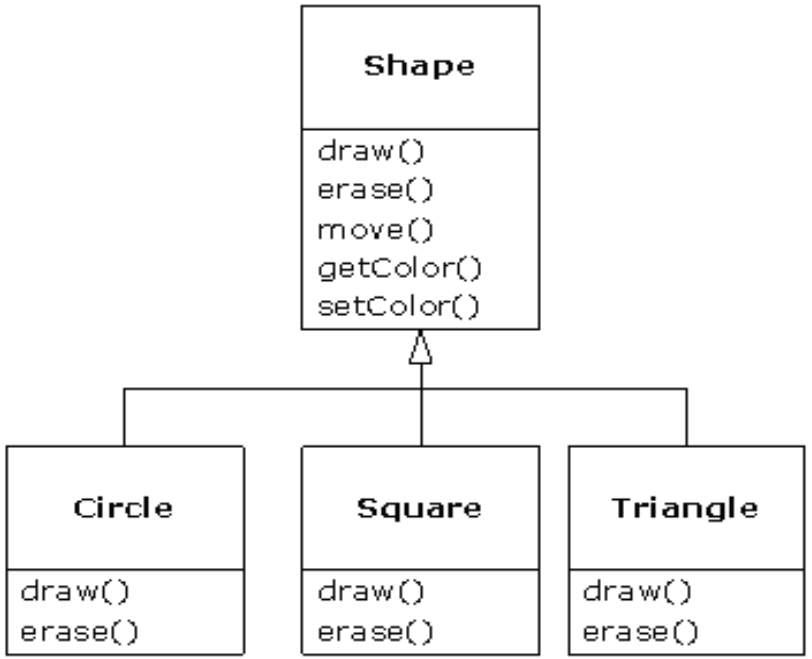
Kế thừa

- Sử dụng lại các giao diện
  và cài đặt
- Các mối quan hệ “is-a”
- Cơ chế cho phép các đối tượng lớp dẫn xuất (derived class) mang đặc điểm (thuộc tính)
  và hành vi (phương thức) của lớp cơ sở (base class)
- Các dịch vụ tổng quát (generic) có thể được xác định
  và thiết kế trước khi đặc biệt hóa chúng



Đa hình

- Đa hình:
  - Khả năng được xem xét ở các dạng thức (forms / shapes) khác nhau
  - Có thể tồn tại ở nhiều dạng
- Đa hình đối tượng:
  - Đối tượng của các lớp dẫn xuất khác nhau có thể được xem như thể hiện của cùng một lớp chung (lớp cơ sở)
  - Các đối tượng của các lớp khác nhau có thể hiểu cùng một thông điệp theo các cách khác nhau
    - Ví dụ: khi nhận thông điệp draw(), các đối tượng Rectangle và Triangle hiểu và thực thi thông điệp theo cách khác nhau



Python vs. Java

Đặc điểm	Python (Tư duy tính toán)	Java (Lập trình nâng cao)
Kiểu dữ liệu (Typing)	**Dynamic (Động):** x = 10 (Tự hiểu kiểu). Linh hoạt nhưng dễ lỗi khi hệ thống lớn.	**Static (Tĩnh):** int x = 10; (Khai báo rõ). Chặt chẽ, giúp IDE phát hiện lỗi ngay khi gõ.
Thực thi (Execution)	**Interpreted (Thông dịch):** Chạy trực tiếp từng dòng. Tiện lợi nhưng chậm hơn.	**Compiled to Bytecode (Biên dịch):** Chạy trên JVM. “Write Once, Run Anywhere”, tối ưu hiệu năng cao.
Quản lý bộ nhớ	**Ẩn (Abstraction cao):** Bạn ít khi phải quan tâm đến vùng nhớ.	**Tường minh hơn:** Phân biệt rõ **Stack** (biến cục bộ) và **Heap** (đối tượng). Kiểm soát tài nguyên tốt hơn.
Cấu trúc mã nguồn	**Indentation:** Dùng thụt lề để phân khối.	**Braces & Classes:** Dùng {} và bắt buộc mọi thứ nằm trong Class. Ép buộc tính mô-đun hóa.
Mục tiêu cốt lõi	Ưu tiên **Tốc độ viết code** (Development Speed).	Ưu tiên **Tính bền bỉ & Bảo trì** (Maintainability & Scale).